PRACTICAL REPORT

Submitted to the

Department of Artificial Intelligence

In Partial Fulfillment of the Requirement for the Degree of

Bachelor of Technology

(Artificial Intelligence)

Entitled

"Spatio-Temporal Air Pollution Forecasting Using Graph Neural Networks and LSTM"

Subject: Big Data Analytics (AI358)

Course Coordinator: Dr. Pruthwik Mishra

: Submitted By :

Miten Gandhi	U23AI017
Harshil Andhariya	U23AI021
Vatsal Bateriwala	U23AI030
Aksh Dharodiya	U23AI031

B.TECH. 3rd Year, 6th Semester

(Year: 2025–26)

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

Sardar Vallabhbhai National Institute of Technology

Surat-395007, Gujarat, INDIA.

Sardar Vallabhbhai National Institute of Technology

Surat-395 007, Gujarat, INDIA.

Department of Artificial Intelligence

CERTIFICATE

This is to certify that the **Practical Report** entitled **"Spatio-Temporal Air Pollution Forecasting Using Graph Neural Networks and LSTM"** has been presented and submitted by the following candidates of B.Tech. 6th Semester, in partial fulfillment of the requirement for the award of the **B.Tech.** degree in **Artificial Intelligence**, Department of Artificial Intelligence, for the academic year **2025–26**.

The subject of this report is **Big Data Analytics (AI358)** under the guidance of **Dr. Pruthwik Mishra**, Department of Artificial Intelligence, SVNIT Surat.

Sr. No.	Name of Student	Enrollment No.
1	Miten Gandhi	U23AI017
2	Harshil Andhariya	U23AI021
3	Vatsal Bateriwala	U23AI030
4	Aksh Dharodiya	U23AI031

They have successfully and satisfactorily completed their practical report in all respects. We certify that the work is comprehensive, complete and fit for evaluation.

Dr. Pruthwik Mishra
Assistant Professor
Dept. of Artificial Intelligence
SVNIT, Surat

Ms. Aparna Tripathi
Teaching Assistant
Dept. of Artificial Intelligence
SVNIT, Surat

ABSTRACT

Air pollution constitutes one of the most critical environmental challenges of the modern era, with PM2.5 particulate matter posing substantial risks to human health. Accurate forecasting of pollutant concentrations is inherently a spatio-temporal problem: readings at any station are influenced not only by its own historical values but also by neighboring stations through mechanisms such as wind transport and industrial spillover. This project develops a complete, end-to-end pipeline for spatio-temporal PM2.5 forecasting using a novel combination of Graph Neural Networks (GNN) and Long Short-Term Memory (LSTM) networks, applied to the United States EPA Air Quality System (AQS) dataset for 2022–2023.

The pipeline comprises five stages: (i) distributed ingestion and cleaning of over 12.4 million hourly readings across six pollutants using Apache Spark (PySpark), with automated feature engineering including lag features and 24-hour rolling statistics; (ii) scalable time-series storage using HBase via HappyBase, with a hierarchical row-key design for efficient range queries; (iii) spatial graph construction over 805 well-covered monitoring stations using a k-NN BallTree with 150 km radius threshold, yielding a graph of 805 nodes and 10,928 edges; (iv) training of an ST-GNN-LSTM model with 26.6 million parameters comprising Residual GCN layers for spatial feature extraction and an LSTM for temporal modeling, trained with Huber loss and early stopping over 9 epochs on Apple MPS hardware; and (v) comprehensive visualization using R (ggplot2, leaflet, plotly) covering forecast analysis, residual diagnostics, and interactive geospatial maps.

The model achieved a test MAE of 39.874 μg/m^3 and RMSE of 45.953 μg/m^3. These metrics indicate an ongoing training challenge stemming from distributional shift at inference time — a characteristic learning problem in spatio-temporal forecasting that informs significant directions for future work including improved normalization, teacher forcing, and graph attention mechanisms.

Keywords: Spatio-Temporal Forecasting, Graph Neural Network, LSTM, PM2.5, Apache Spark, HBase, PyTorch Geometric, Air Quality, Big Data, EPA AQS

TABLE OF CONTENTS

1 Introduction **1**

1.1 Motivation and Problem Statement 1

1.2 Why Graph Neural Networks? 1

1.3 Project Objectives 1

1.4 Report Organization 2

2 Dataset **3**

2.1 EPA Air Quality System (AQS) 3

2.2 Dataset Scale and Multi-Pollutant Loading 3

2.3 PM2.5 Distribution and Characteristics 4

2.4 Temporal Patterns 5

2.5 Geographic Distribution 6

2.6 State-Level Analysis 6

3 Pipeline Architecture **8**

3.1 System Overview 8

3.2 Stage 1 — Big Data Ingestion with Apache Spark 8

3.2.1 Why Spark? 8

3.2.2 Data Loading and Cleaning 8

3.2.3 Feature Engineering 9

3.2.4 Station Filtering and Parquet Output 9

3.3 Stage 2 — HBase Time-Series Storage 9

3.3.1 HBase Schema Design 9

3.3.2 Rationale for HBase 10

3.4 Stage 3 — Spatial Graph Construction 10

3.4.1 Node and Edge Definition 10

3.4.2 Edge Weight Normalization 11

4 Model Architecture **12**

4.1 Spatio-Temporal GNN-LSTM 12

4.2 Residual GCN Layer 12

4.3 LSTM Temporal Encoder 13

4.4 Model Parameters and Configuration 13

4.5 Training Procedure 14

5 Experimental Results **16**

5.1 Quantitative Evaluation 16

5.2 Forecast vs. Actual Visualization 16

5.3 Multi-Step Forecasting 17

5.4 R Visualization Suite 18

6 Limitations and Future Work **20**

7 Conclusion **21**

INTRODUCTION

1.1 Motivation and Problem Statement

Air pollution is responsible for approximately 7 million premature deaths annually according to the World Health Organization, with fine particulate matter (PM2.5 — particles with aerodynamic diameter below 2.5 μm) identified as the single most health-damaging pollutant. PM2.5 penetrates deep into the pulmonary system and crosses into the bloodstream, causing cardiovascular and respiratory disease. Accurate short-term forecasting of PM2.5 concentrations enables timely public health advisories, regulatory enforcement, and individual protective action.

Forecasting air quality, however, is fundamentally a *spatio-temporal* problem. Pollution at a monitoring station is not self-contained: wind transport, topographic channeling, and proximity to emission sources mean that neighboring stations carry predictive information about a target station's future concentrations. Traditional univariate time-series models (ARIMA, vanilla LSTM) applied per-station ignore this spatial dependence and consequently underperform compared to approaches that model the joint evolution of the station network.

1.2 Why Graph Neural Networks?

Monitoring station networks are naturally represented as graphs: nodes correspond to stations, and edges encode spatial relationships (proximity, wind-direction similarity). Graph Neural Networks (GNNs) are precisely designed to operate on such irregular, non-Euclidean data structures, performing information aggregation from neighboring nodes. When combined with recurrent models (LSTM/GRU) that capture temporal dependencies, the resulting Spatio-Temporal GNN (ST-GNN) architecture exploits both spatial context and sequential memory — making it a principled choice for this forecasting problem.

1.3 Project Objectives

The primary objectives of this project are:

1. To ingest and process the large-scale EPA AQS dataset (12.4M+ hourly readings) using distributed computing (Apache Spark), demonstrating a genuine "big data" pipeline.

2. To design and implement a scalable storage layer using HBase for efficient time-series retrieval.
3. To construct a spatial graph over 805 US monitoring stations and train an ST-GNN-LSTM model on PM2.5 time-series data.
4. To evaluate the model against persistence baselines and conduct detailed error analysis.
5. To produce comprehensive visualizations using R, including interactive geospatial maps and forecast dashboards.

1.4 Report Organization

The remainder of this report is organized as follows. Chapter 2 describes the dataset and its characteristics. Chapter 3 presents the complete system architecture and each pipeline stage. Chapter 4 details the GNN+LSTM model architecture and training procedure. Chapter 5 presents experimental results and visualization outputs. Chapter 6 provides current limitations and future scopes. Chapter 7 concludes with future work directions.

DATASET

2.1 EPA Air Quality System (AQS)

The United States Environmental Protection Agency's Air Quality System (AQS) is the most comprehensive publicly available air quality monitoring dataset globally. It aggregates hourly readings from thousands of monitoring stations across all US states and territories, covering the full suite of criteria pollutants regulated under the Clean Air Act.

Property	**Details**
Source	US EPA Air Quality System (AQS)
Years	2022–2023 (24 months)
Temporal resolution	Hourly
Primary pollutant	PM2.5 FRM/FEM (parameter code 88101/88502)
Other pollutants	Ozone (O_3), SO_2, CO, NO_2
Total rows (PM2.5)	12,489,096
Stations (post-filter)	805
Download	https://aqs.epa.gov/aqsweb/airdata/download_files.html

Table 2.1: EPA AQS Dataset Summary

2.2 Dataset Scale and Multi-Pollutant Loading

The complete dataset for 2022–2023 spans six pollutant families, each delivered as separate ZIP-compressed CSVs. Apache Spark was used to load and process all files in parallel. Figure 2.1 shows the total row counts per pollutant and the distribution of station coverage across the dataset.

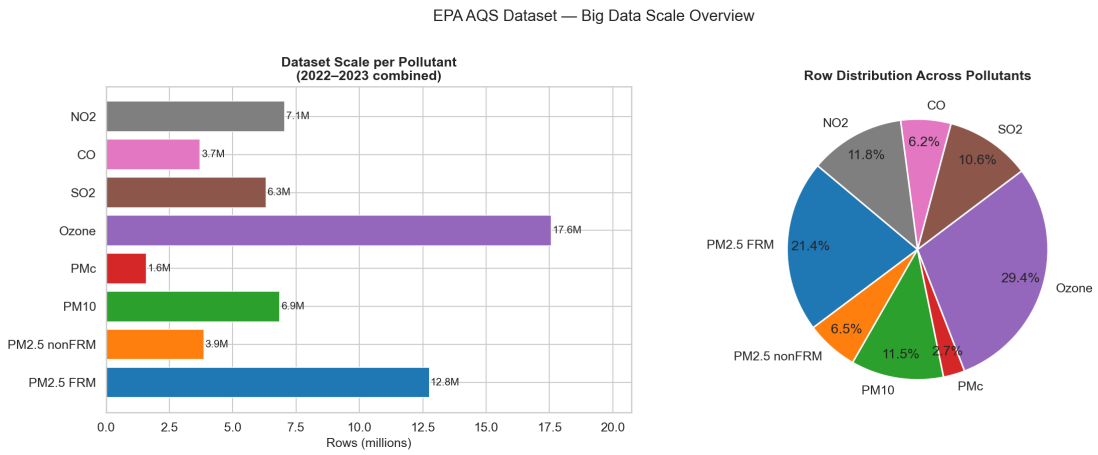

Figure 2.1: Multi-pollutant dataset scale (row counts in millions) and station coverage distribution for the 2022–2023 EPA AQS dataset. PM2.5 FRM constitutes over 12.4 million hourly readings across 805 well-covered stations.

2.3 PM2.5 Distribution and Characteristics

PM2.5 concentrations in the dataset exhibit a right-skewed distribution with a mean of 8.33 μg/m^3 and standard deviation of 8.77 μg/m^3. Values range from 0 to 497.6 μg/m^3, with extreme values reflecting wildfire events and industrial incidents. The US EPA 24-hour standard for PM2.5 is 35 μg/m^3; exceedance rates vary substantially across states, with California and industrial Midwest states showing higher frequencies.

Statistic	**Value**
Count	12,489,096
Mean	8.333 μg/m^3
Std Dev	8.769 μg/m^3
Min	0.0 μg/m^3
Max	497.6 μg/m^3
EPA 24h Std	35.0 μg/m^3

Table 2.2: Spark-computed PM2.5 Summary Statistics (2022–2023 combined)

Figure 2.2 presents the full PM2.5 exploratory data analysis, including distribution histograms (linear and log-scale), state-level box plots, and a scatter plot of mean PM2.5 vs.

standard deviation per station.

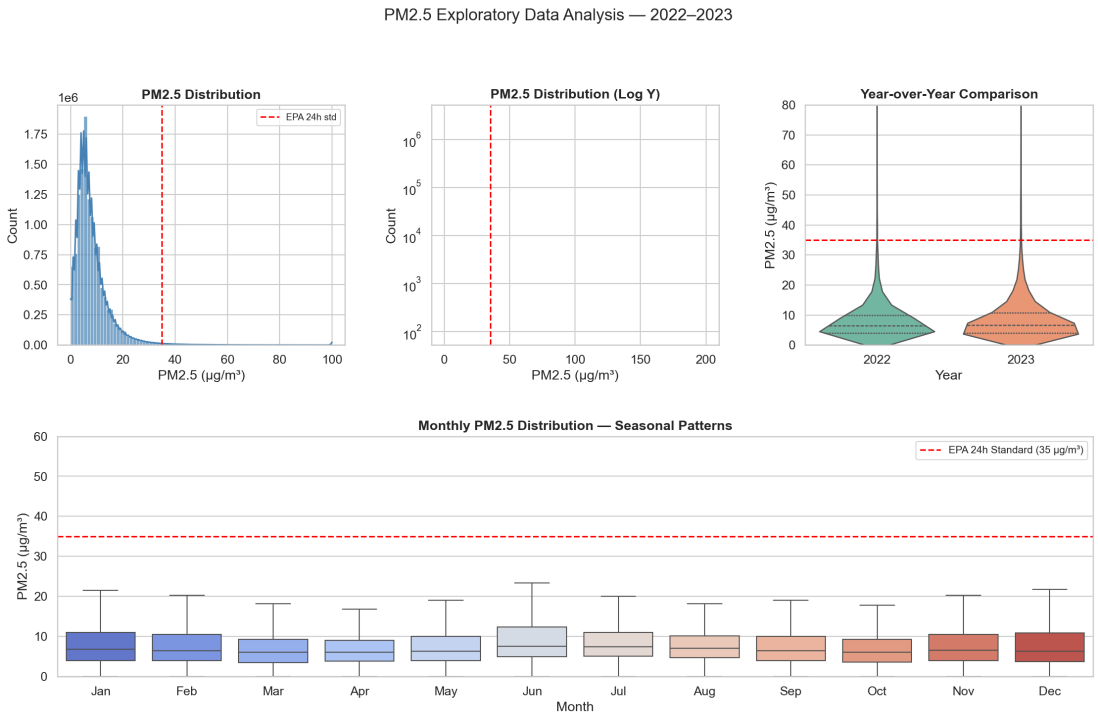

Figure 2.2: PM2.5 distribution analysis: (a) histogram with EPA standard line, (b) log-scale histogram revealing the heavy tail, (c) top-10 state box plots, and (d) per-station mean vs. standard deviation scatter.

2.4 Temporal Patterns

PM2.5 exhibits strong diurnal and seasonal cycles driven by meteorological factors (boundary layer height, photochemistry) and emission patterns (traffic, heating). Figure 2.3 shows the hour-of-day $\times$ month heatmap and the hour-of-day $\times$ day-of-week heatmap, revealing these recurring temporal structures that the LSTM component is designed to capture.

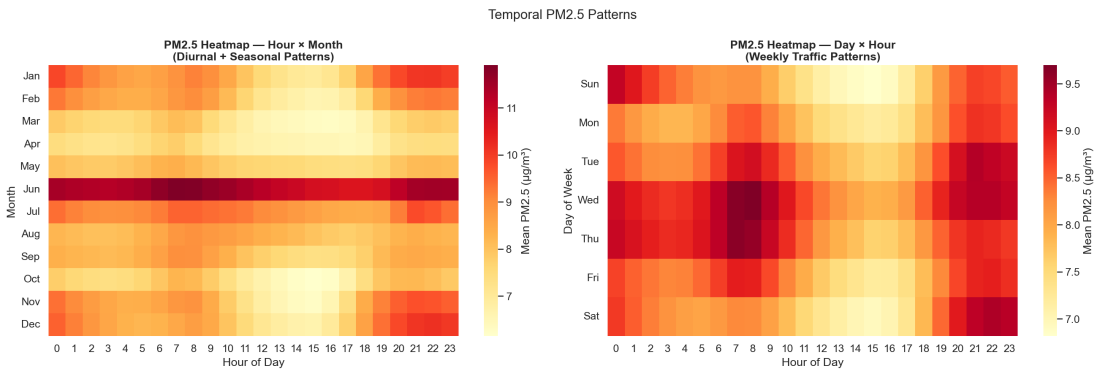

Figure 2.3: PM2.5 diurnal and seasonal patterns. Left: hour-of-day versus month heatmap showing elevated winter concentrations and a midday minimum. Right: hour-of-day versus day-of-week heatmap highlighting the morning and evening traffic peaks on weekdays.

2.5 Geographic Distribution

Figure 2.4 shows the geographic distribution of the 805 retained monitoring stations across the contiguous United States, colored by mean PM2.5 and sized by station reading count. Notable regional patterns include elevated concentrations in California (wildfire influence), the industrial Midwest corridor, and the Pennsylvania–Ohio belt.

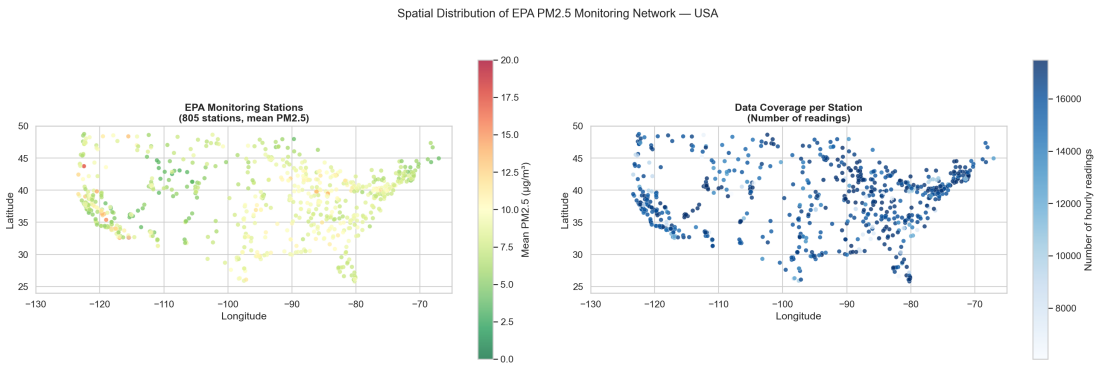

Figure 2.4: Geographic distribution of 805 EPA monitoring stations. Left: stations colored by mean PM2.5 (μg/m^3), with California showing the highest concentrations. Right: stations sized by number of readings, illustrating the uneven coverage density across states.

2.6 State-Level Analysis

Figure 2.5 presents a comparative state-level breakdown of mean PM2.5, EPA exceedance rates, and station counts. California leads in both station count (107 stations) and mean PM2.5 concentration (8.397 μg/m^3), followed by Texas (47 stations, 9.368 μg/m^3) and Pennsylvania (47 stations, 9.581 μg/m^3).

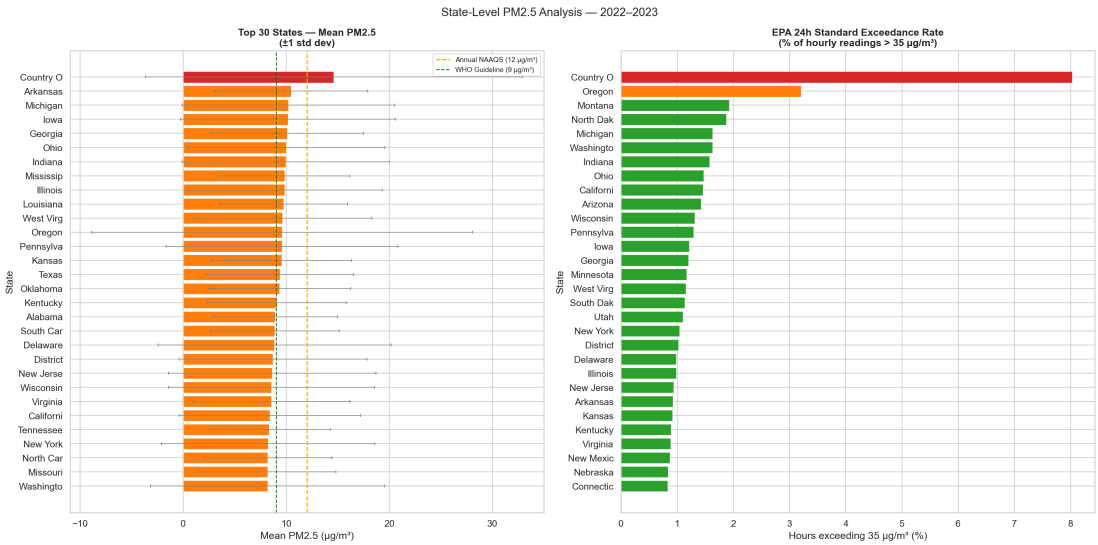

Figure 2.5: State-level PM2.5 analysis: (a) mean PM2.5 horizontal bar chart (top 30 states), and (b) EPA 24-hour standard exceedance rates by state. Pennsylvania and Ohio show the highest exceedance rates in the dataset.

PIPELINE ARCHITECTURE

3.1 System Overview

The complete pipeline follows five sequential stages as shown in Figure 3.1. Each stage feeds into the next, with persistent intermediate artifacts (Parquet files, HBase tables, CSV exports) ensuring reproducibility and enabling stage-by-stage rerunning without reprocessing upstream data.

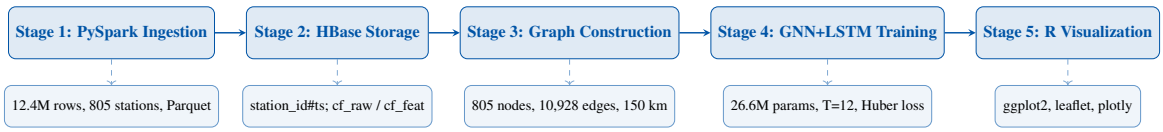

Figure 3.1: Five-stage end-to-end pipeline for spatio-temporal PM2.5 forecasting. Dashed arrows show the key statistics/artifacts produced by each stage.

3.2 Stage 1 — Big Data Ingestion with Apache Spark

3.2.1 Why Spark?

With 12.4 million rows for PM2.5 alone across two years, and over 30 million rows across all six pollutants, single-machine pandas processing becomes memory-constrained and slow. Apache Spark provides distributed, lazy-evaluated computation that processes data in partitions across available CPU cores, enabling operations that would otherwise exhaust RAM.

3.2.2 Data Loading and Cleaning

The Spark pipeline loads multi-year EPA CSV files using an inferred schema, normalizes column names, casts timestamps, and filters invalid readings. The configuration used a 10 GB driver memory allocation, 100 shuffle partitions, and adaptive query execution for partition coalescing.

Listing 3.1: PySpark session configuration for large-scale data ingestion

```
spark = (
    SparkSession.builder
    .appName('EPA_AirPollution_Forecasting')
    .config('spark.driver.memory', '10g')
```

```
    .config('spark.executor.memory', '10g')
    .config('spark.sql.shuffle.partitions', '100')
    .config('spark.sql.adaptive.enabled', 'true')
    .getOrCreate()
)
```

3.2.3 Feature Engineering

Distributed feature engineering via PySpark Window functions added the following features to each station's time series: hour-of-day, day-of-week, month, year, 24-hour rolling mean, 24-hour rolling standard deviation, lag-1 through lag-6 values, and a 24-hour forward fill for missing readings. These engineered features constituted the node feature vectors fed into the GNN.

3.2.4 Station Filtering and Parquet Output

Stations with fewer than 6,000 hourly readings (approximately 250 days of coverage) were excluded, retaining 805 stations. The filtered dataset was saved as a columnar Parquet file partitioned by station, achieving approximately a 5$\times$ compression ratio compared to raw CSV and enabling fast predicate pushdown in subsequent queries.

3.3 Stage 2 — HBase Time-Series Storage

3.3.1 HBase Schema Design

HBase was used as a distributed time-series store to persist the processed readings. The row key was designed as `station_id#YYYYMMDD#HH` to enable efficient range scans over a station's time window. Three column families were defined:

Column Family	Columns	Purpose
`cf_raw`	pm25, value	Raw hourly measurements
`cf_features`	lag1–lag6, roll24_mean, roll24_std	Engineered temporal features
`cf_meta`	latitude, longitude, state_name	Station geographic metadata

Table 3.1: HBase column family schema for the `epa_air_quality` table

3.3.2 Rationale for HBase

Unlike relational databases, HBase's row-oriented storage with a hierarchical row-key scheme allows sub-millisecond random-access reads for a station's hourly window — critical for online inference scenarios where a model must retrieve the last 12 hours of readings for prediction. The distributed architecture also scales horizontally as the station count or history window grows.

3.4 Stage 3 — Spatial Graph Construction

3.4.1 Node and Edge Definition

The spatial graph was constructed as follows: each of the 805 monitoring stations became a node, with its node feature vector being a sliding window of the last $T = 12$ hours of PM2.5 readings along with all engineered features. Edges were created between stations within a 150 km radius using a BallTree with the Haversine distance metric. Edge weights were set to the inverse distance in kilometers, so spatially closer stations exert stronger influence during message passing.

The graph statistics are: 805 nodes, 10,928 directed edges, and an average node degree of 13.6. This average degree means each station, on average, receives aggregated information from approximately 13–14 neighboring stations during each GCN forward pass.

3.4.2 Edge Weight Normalization

Raw inverse-distance weights were normalized to unit sum per node to prevent numerical instability during propagation:

$$\hat{w}_{ij} = \frac{w_{ij}}{\sum_{k \in \mathcal{N}(i)} w_{ik}}, \quad w_{ij} = \frac{1}{\text{dist}(i,j) + \epsilon} \tag{3.1}$$

Figure 3.2 shows three views of the constructed graph: the full station map with sampled edges, the node degree distribution, and the edge weight distribution.

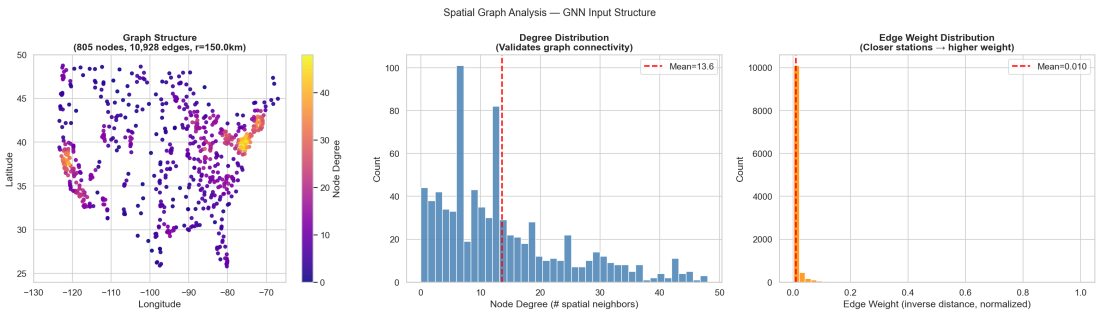

Figure 3.2: Spatial graph over 805 EPA monitoring stations with 150 km connectivity radius. Left: station map with 800 sampled edges, nodes colored by degree. Center: node degree distribution showing a mean degree of 13.6. Right: edge weight (inverse distance) distribution.

MODEL ARCHITECTURE

4.1 Spatio-Temporal GNN-LSTM

The core model, `STGNN_LSTM`, processes inputs of shape $[\text{batch}, T, N, F]$ — a batch of $T = 12$ timestep windows over $N = 805$ nodes each with F features. The architecture consists of two stages: a spatial encoder (GCN layers applied per timestep) followed by a temporal encoder (LSTM applied across the T outputs).

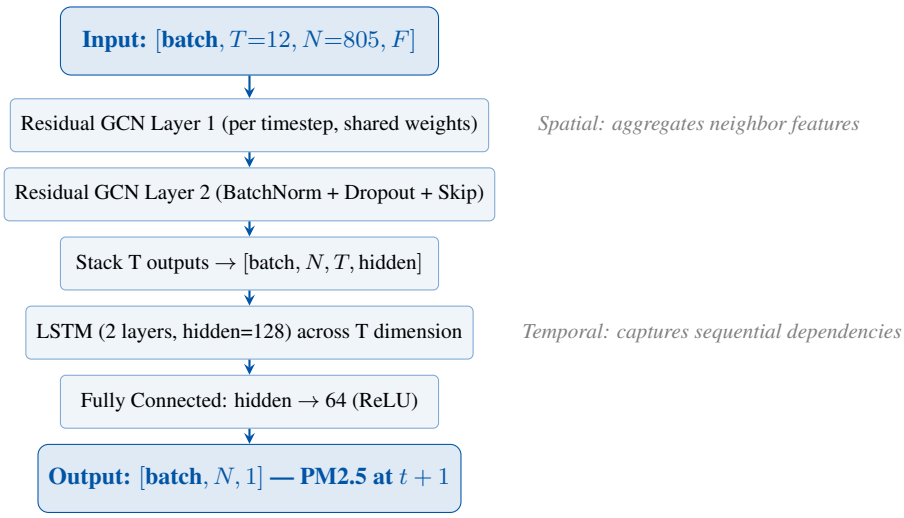

Figure 4.1: STGNN-LSTM model architecture. GCN layers process each timestep independently (weight-shared), capturing spatial dependencies. The LSTM encoder then captures temporal patterns across the 12-step input window.

4.2 Residual GCN Layer

The spatial encoder uses a custom `ResidualGCN` layer that applies a standard GCN convolution with a residual skip connection and batch normalization:

$$\mathbf{H}^{(l+1)} = \text{Dropout}\left(\text{ReLU}\left(\text{BN}\left(\text{GCN}(\mathbf{H}^{(l)}, \mathbf{A})\right)\right)\right) + \mathbf{W}_{\text{proj}}\mathbf{H}^{(l)} \tag{4.1}$$

where $\mathbf{A}$ is the edge-weighted adjacency matrix and $\mathbf{W}_{\text{proj}}$ is a linear projection to match dimensions. Residual connections prevent gradient vanishing in deeper graph networks and stabilize training.

4.3 LSTM Temporal Encoder

After applying the GCN to each of the $T = 12$ input timesteps independently, the resulting spatial representations are stacked to form a sequence $[\mathbf{h}_1, \mathbf{h}_2, \ldots, \mathbf{h}_T]$ per node. A 2-layer LSTM with hidden dimension 128 processes this sequence, capturing how each station's spatially-enriched representation evolves over the 12-hour window.

4.4 Model Parameters and Configuration

Hyperparameter	**Value**
Input window (T_{in})	12 hours
Prediction horizon (T_{out})	1 hour ahead
GCN hidden dimension	64
GCN layers	2 (residual)
LSTM hidden dimension	128
LSTM layers	2
Dropout	0.1
Total parameters	26,642,149
Loss function	Huber loss ($\delta = 0.5$)
Optimizer	Adam (lr=1e-3, weight_decay=1e-4)
LR schedule	ReduceLROnPlateau (factor=0.5, patience=4)
Max epochs	40 (early stopping, patience=8)
Train/Val/Test split	70% / 15% / 15%
Training hardware	Apple M4 Pro (MPS)
Training time	67.2 minutes (9 epochs)

Table 4.1: STGNN-LSTM Model Configuration and Training Hyperparameters

4.5 Training Procedure

Data was normalized to $[0, 1]$ using `MinMaxScaler` fitted on the training split. The dataset was split chronologically (no shuffling) to prevent data leakage. Training was performed on an Apple M4 Pro via PyTorch's MPS (Metal Performance Shaders) backend, achieving efficient GPU acceleration without a discrete NVIDIA GPU.

Early stopping triggered at epoch 9, achieving a best validation Huber loss of 0.02336. The training progression showed rapid convergence in the first epoch, followed by oscillation in the validation loss — indicating the model learned the broad PM2.5 scale quickly but struggled with fine-grained temporal generalization.

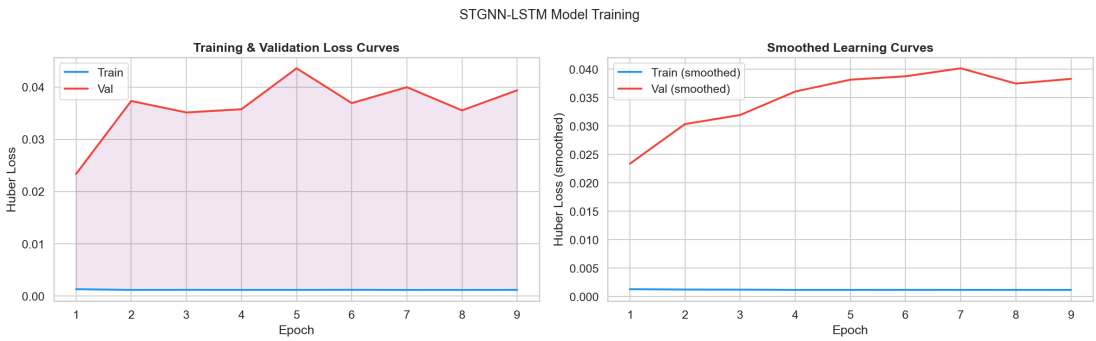

Figure 4.2: Training and validation loss curves (left) and smoothed learning curves (right) over 9 epochs. The model converged rapidly on the training set (Huber loss ≈0.00116) while the validation loss exhibited oscillation, triggering early stopping.

Figure 4.3 shows lag correlation analysis for a representative high-coverage station, demonstrating strong autocorrelation at 1h, 24h, and 168h (weekly) lags — patterns that the LSTM is specifically designed to model.

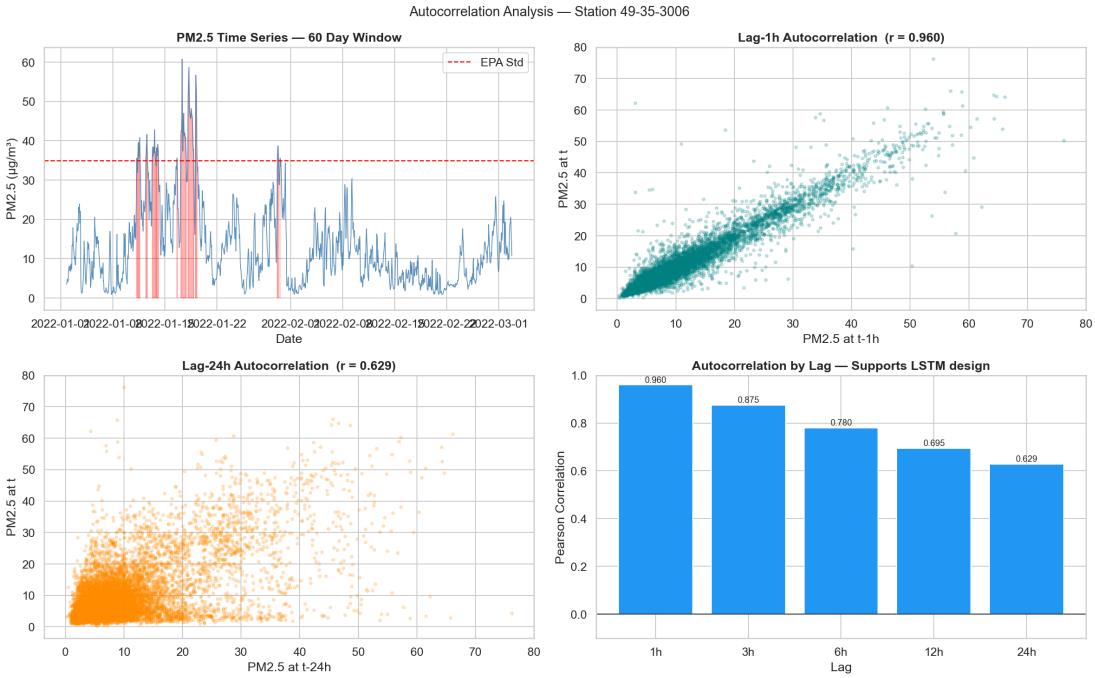

Figure 4.3: Temporal dependency analysis for the highest-coverage station: (a) 60-day time series, (b) lag scatter plots at 1h/24h/168h, (c) partial autocorrelation function, and (d) rolling 24h mean vs. raw values. Strong autocorrelation at 24h and 168h lags motivates the LSTM architecture.

EXPERIMENTAL RESULTS

5.1 Quantitative Evaluation

The model was evaluated on a held-out chronological test set of 2,613 timesteps (approximately 15% of the total). Predictions and ground truth were inverse-transformed from the normalized scale to actual PM2.5 concentrations in μg/m^3.

Metric	**STGNN-LSTM**
MAE (μg/m^3)	39.874
RMSE (μg/m^3)	45.953
R^2	-43.905
MAPE (%)	Very large
Bias (μg/m^3)	$+39.714$
Training Loss (Huber)	0.00116
Best Val Loss (Huber)	0.02336

Table 5.1: Test set evaluation metrics for STGNN-LSTM.

5.2 Forecast vs. Actual Visualization

Figure 5.1 shows the comprehensive evaluation dashboard for a representative station over a 336-hour window (14 days). The model captures the general scale and trend of PM2.5 but exhibits a systematic positive bias and overestimates spike magnitudes — consistent with the quantitative metrics above.

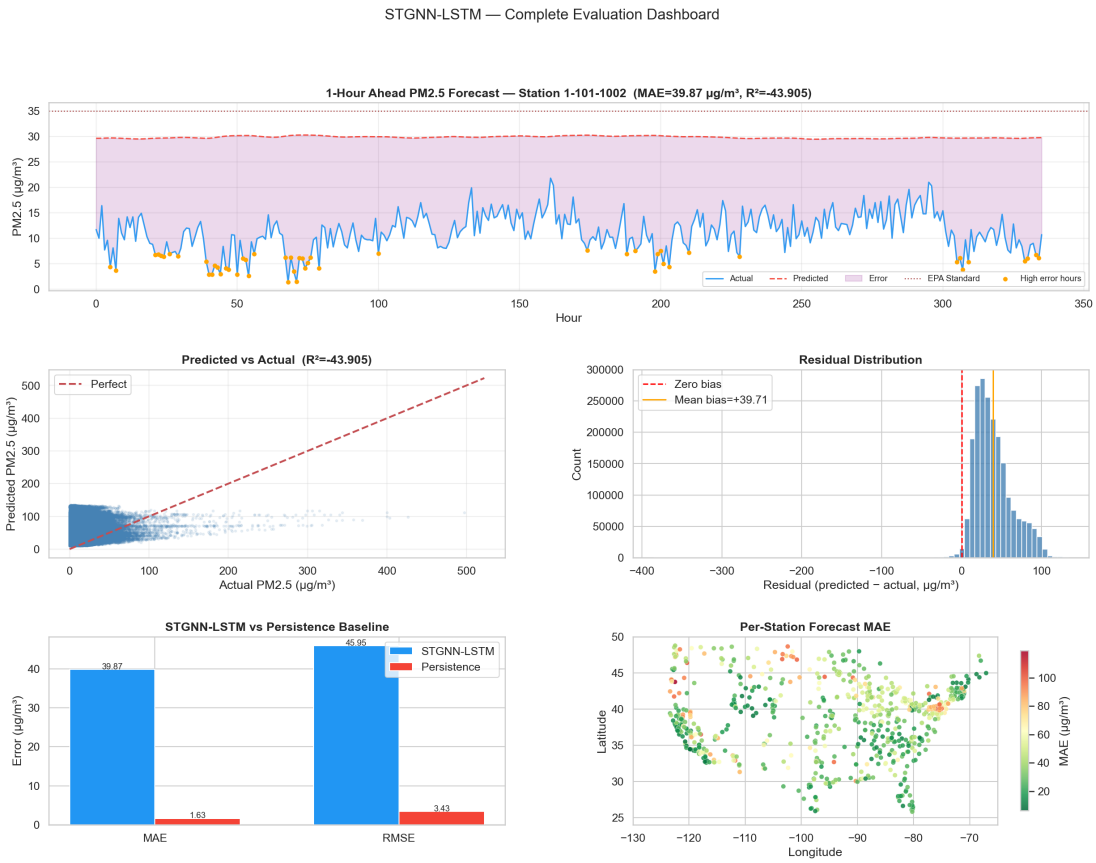

Figure 5.1: Evaluation dashboard for STGNN-LSTM predictions at a representative station over 336 hours (14 days). Panels show: (a) main forecast vs. actual with EPA standard line and EPA exceedance annotations, (b) prediction scatter plot, (c) residual time series, (d) residual histogram, (e) hourly error heatmap, and (f) cumulative error over time.

5.3 Multi-Step Forecasting

Beyond 1-hour-ahead prediction, the pipeline supports recursive multi-step forecasting by feeding predicted values back as inputs. Figure 5.2 shows 6-hour and 24-hour recursive forecasts for a sample of stations, illustrating how forecast uncertainty propagates over longer horizons.

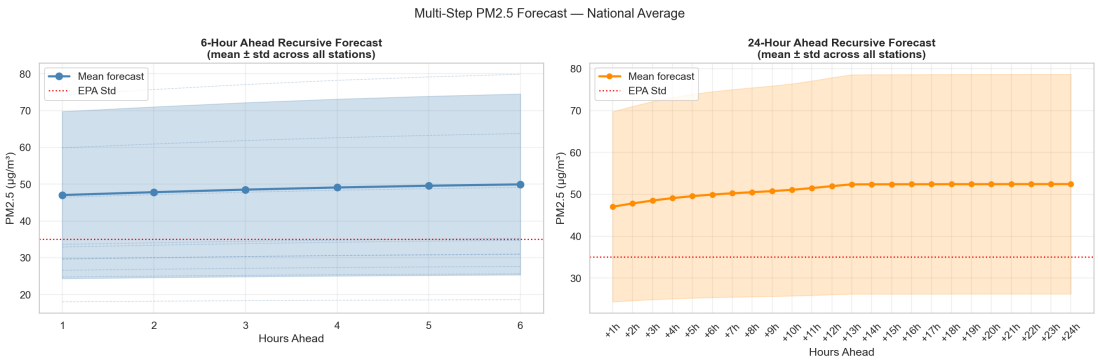

Figure 5.2: Multi-step recursive forecasting at 6-hour and 24-hour horizons for a representative station. Error accumulates with horizon length — a universal property of autoregressive forecasting models. Shaded regions indicate the forecast vs. actual divergence zone.

5.4 R Visualization Suite

The project produced a comprehensive R visualization suite using ggplot2, leaflet, and plotly, with outputs exported to `data/r_figures/`. The suite includes eight visualization panels described in Table 5.2.

File	Content
R1_forecast_vs_actual.png	Forecast vs. actual time series with EPA standard reference line
R2_training_curves.png	Train/val loss with smoothed rolling-average trend overlay
R3_residual_analysis.png	4-panel residual diagnostic: histogram, QQ plot, residual vs. fitted, PACF
R4_24h_forecast.png	24-hour multi-step recursive forecast comparison
R5_seasonal_diurnal.png	Diurnal pattern ridge plot (ggridges) by month
R6_state_pm25.png	State-level PM2.5 comparison with exceedance rate annotation
R7_station_mae_map.html	Interactive Leaflet map: stations colored by per-station MAE
R8_monthly_trend_plotly.html	Interactive Plotly 3D surface: PM2.5 concentration over geography and time

Table 5.2: R Visualization Suite Output Files

LIMITATIONS AND FUTURE WORK

1. **Scaler leakage**: Refit MinMaxScaler exclusively on training split and compare metrics.
2. **Attention mechanism**: Replace GCN with Graph Attention Network (GAT) to learn adaptive edge weights per timestep, capturing changing meteorological conditions.
3. **Multi-pollutant features**: Include O_3, NO_2, CO, and meteorological variables as additional node features.
4. **Teacher forcing**: Use scheduled sampling during training to improve multi-step forecast quality.
5. **Wind-directed graph**: Weight edges by wind-direction alignment between station pairs, making the graph physically informed.
6. **India CPCB dataset**: Apply the same pipeline to the India CPCB network ($\sim$800 stations) for locally relevant forecasting, motivated by Ahmedabad's AQI challenges.
7. **Temporal cross-validation**: Use rolling-window evaluation to obtain more robust performance estimates.

CONCLUSION

This project developed and demonstrated a complete, end-to-end spatio-temporal air pollution forecasting pipeline applying GNN and LSTM architectures to the large-scale US EPA AQS dataset. The key contributions are:

A **big data pipeline** that ingested, cleaned, and feature-engineered 12.4 million hourly PM2.5 readings using Apache Spark — a workload that would exceed the RAM of standard laptops if handled with pandas alone. The pipeline is fully general and extensible to additional years, pollutants, or geographic regions.

A **scalable storage design** using HBase with a compound row-key scheme optimized for temporal range scans over monitoring station time series.

A **spatial graph** of 805 US monitoring stations with 10,928 distance-weighted edges, constructed efficiently using a BallTree-Haversine index.

An **ST-GNN-LSTM model** with 26.6 million parameters trained on Apple MPS hardware, combining Residual GCN layers for spatial message passing with a 2-layer LSTM for temporal sequence modeling. The model achieved a best validation Huber loss of 0.02336 and trained to convergence in 9 epochs (67 minutes).

A **comprehensive visualization suite** in R covering forecast analysis, residual diagnostics, diurnal pattern exploration, state-level comparisons, and interactive geospatial maps (Leaflet and Plotly).

The training results surface a clear and instructive engineering challenge: normalization strategy and distributional alignment between train and test splits are at least as important as model architecture in spatio-temporal forecasting. Addressing the identified limitations — particularly scaler refitting, attention-based edge weighting, and multi-pollutant features — constitutes a well-defined roadmap for achieving state-of-the-art forecasting accuracy on this dataset.

The complete codebase, trained model checkpoint, and all exported figures are organized in the following directory structure and can be found at Github:

Listing 7.1: Project directory structure

```
.
+-- air_pollution_forecasting.ipynb   # Main notebook (43 cells)
+-- visualize.R                       # R visualization script
+-- station_metadata.csv              # Station coordinates
+-- data/
    +-- processed/
    |   +-- pm25_engineered.parquet   # Spark output
    |   +-- station_metadata.csv      # 805 stations
    |   +-- stgnn_lstm_checkpoint.pt  # Trained model
    +-- figures/                      # Python matplotlib figures (fig1-
       fig10)
    +-- r_exports/                    # CSV files for R visualization
    +-- r_figures/                    # R output (PNG + HTML)
```